

1. Define JSX

JSX stands for JavaScript XML. It is a syntax extension for JavaScript that allows you to write HTML-like code directly inside JavaScript. JSX makes it easier to write and understand React components because the UI structure looks similar to HTML.

JSX is not valid JavaScript on its own. It needs to be compiled by tools like Babel into regular JavaScript that browsers can understand. When you write JSX, it gets transformed into `React.createElement()` calls.

Example of JSX:

```
text
const element = <h1>Hello, World!</h1>;
```

This looks like HTML but it is actually JavaScript. The advantage of JSX is that you can write your UI structure and logic in one place, making the code more readable and maintainable.

2. Explain about ECMA Script

ECMAScript is the standard on which JavaScript is based. It is a scripting language specification that defines the rules and features of JavaScript. ECMAScript is maintained by ECMA International, an organization that creates standards for technologies.

JavaScript is the implementation of ECMAScript. When people talk about ES6, ES7, ES8, they are referring to different versions of ECMAScript. Each new version adds new features and improvements to the language.

Some important ECMAScript versions:

ES6 (ECMAScript 2015) introduced major features like arrow functions, classes, template literals, destructuring, spread operator, and modules.

ES7 (ECMAScript 2016) added features like `Array.includes()` and exponentiation operator.

ES8 (ECMAScript 2017) added `async/await`, `Object.values()`, and `Object.entries()`.

ES9 (ECMAScript 2018) added rest/spread properties and asynchronous iteration.

ES10 (ECMAScript 2019) added `Array.flat()` and `Object.fromEntries()`.

React heavily relies on modern ECMAScript features, especially ES6. When building React applications, you will commonly use arrow functions, classes, destructuring, spread operator, and modules.

3. Explain `React.createElement()`

`React.createElement()` is the core function that React uses to create elements. It is the underlying mechanism that powers JSX. When you write JSX, it gets compiled into `React.createElement()` calls.

The syntax of `React.createElement` is:

`React.createElement(type, props, children)`

Type is the element type, which can be a string for HTML tags like `div`, `h1`, `p`, or it can be a React component.

Props is an object containing the properties or attributes for the element.

This includes things like `className`, `id`, `style`, and event handlers.

Children is the content inside the element. This can be a string, another element, or an array of elements.

Here is an example of `React.createElement`:

```
text

React.createElement('h1', { className: 'title' }, 'Hello World');
```

This creates an `h1` element with a class of `title` and the text `Hello World`.

When you write JSX like this:

```
text

<h1 className="title">Hello World</h1>
```

It gets compiled to:

```
text

React.createElement('h1', { className: 'title' }, 'Hello World');
```

The benefit of using JSX is that it is more readable and easier to write than `React.createElement` calls directly.

4. Explain how to create React nodes with JSX

Creating React nodes with JSX is straightforward. You simply write HTML-like tags inside your JavaScript code. Every JSX element represents a React node.

To create a simple element:

```
text

const heading = <h1>Welcome to React</h1>;
```

To create an element with attributes:

```
text

const button = <button className="btn-primary">Click Me</button>;
```

To create nested elements:

```
text

const container = (
  <div>
    <h1>Title</h1>
    <p>This is a paragraph</p>
  </div>
);
```

To create elements that represent React components, you use component names with capital letters:

```
text
```

```
const App = <MyComponent name="John" />;
```

Important rules when creating React nodes with JSX:

Every tag must be closed. Self-closing tags like `img`, `input`, and `br` must end with a slash.

```
text
```

```

```

```
<input type="text" />
```

```
<br />
```

Adjacent JSX elements must be wrapped in a parent element or fragment.

```
text
```

```
// This is wrong
```

```
const wrong = <h1>Title</h1><p>Text</p>;
```

```
// This is correct
```

```
const correct = (
```

```
  <div>
```

```
    <h1>Title</h1>
```

```
    <p>Text</p>
```

```
  </div>
```

```
);
```

```
// Or using a fragment
```

```
const fragment = (
```

```
  <>
```

```
    <h1>Title</h1>
```

```
    <p>Text</p>
```

```
  </>
```

```
);
```

JSX elements can be stored in variables, returned from functions, and passed as arguments.

5. Define how to render JSX to DOM

Rendering JSX to the DOM is done using ReactDOM, which is a separate package from React. React is responsible for creating the components and elements, while ReactDOM is responsible for rendering them to the browser DOM.

The entry point for rendering is the root element in your HTML file. This is usually a div with an id of root.

HTML:

```
text

<div id="root"></div>
```

To render a React element to the DOM, you use ReactDOM.createRoot() and then call the render method.

Basic rendering:

```
text

import React from 'react';
import ReactDOM from 'react-dom/client';

const element = <h1>Hello, World!</h1>;
const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(element);
```

In a typical React application, you only call `root.render()` once. This is usually done in the `index.js` file. The App component is rendered at the root level.

text

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));

root.render(<App />);
```

Once rendered, React takes care of updating the DOM whenever the application state changes. React uses its virtual DOM to calculate the most efficient way to update the real DOM.

You can also render multiple times if needed, but this is not common in React applications.

6. Explain how to use JavaScript expressions in JSX

JSX allows you to embed JavaScript expressions inside curly braces. This is one of the most powerful features of JSX because it lets you create dynamic content.

Any valid JavaScript expression can be placed inside curly braces. This includes variables, function calls, arithmetic operations, ternary operators, and more.

Using variables:

```
text
```

```
const name = 'John';
```

```
const greeting = <h1>Hello, {name}!</h1>;
```

Using arithmetic:

```
text
```

```
const total = <p>Total: {10 + 20}</p>;
```

Using function calls:

```
text
```

```
function getGreeting(name) {  
  return 'Hello ' + name;  
}
```

```
const greeting = <h1>{getGreeting('John')}</h1>;
```

Using ternary operators for conditional rendering:

```
text
```

```
const isLoggedIn = true;
```

```
const message = <h1>{isLoggedIn ? 'Welcome back!' : 'Please  
login'}</h1>;
```

Using arrays with map to render lists:

```
text
```

```
const names = ['John', 'Jane', 'Bob'];  
const list = (  
  <ul>  
    {names.map((name, index) => (  
      <li>{name}</li>  
    )}  )
```



```
        <li key={index}>{name}</li>
      )})
    </ul>
```

```
);
```

Using object properties:

```
text
```

```
const user = { name: 'John', age: 25 };
```

```
const userInfo = (
```

```
  <div>
```

```
    <p>Name: {user.name}</p>
```

```
    <p>Age: {user.age}</p>
```

```
  </div>
```

```
);
```

You cannot use statements like if, for, or while inside curly braces. Only expressions that return a value are allowed. For conditional logic, you should use ternary operators or logical AND operators.

Using logical AND for conditional rendering:

```
text
```

```
const showMessage = true;
```

```
const message = <div>{showMessage && <p>This message is
shown</p>}</div>;
```

7. Explain how to use inline CSS in JSX

Inline CSS in JSX is done using the style attribute. Unlike regular HTML where style is a string, in JSX the style attribute accepts a JavaScript object.

The style object uses camelCase property names instead of kebab-case. For example, background-color becomes backgroundColor, font-size becomes fontSize.

Basic inline CSS:

```
text

const heading = <h1 style={{ color: 'blue', fontSize: '24px'
}}>Hello World</h1>;
```

The double curly braces might look confusing. The outer braces are for the JSX expression, and the inner braces are for the JavaScript object.

You can also define the style object separately:

```
text

const styles = {
  color: 'blue',
  fontSize: '24px',
  padding: '10px',
  backgroundColor: '#f0f0f0'
};

const heading = <h1 style={styles}>Hello World</h1>;
```

Using dynamic values in inline CSS:

```
text

const isActive = true;
```

```
const styles = {  
  color: isActive ? 'green' : 'red',  
  fontWeight: 'bold'  
};
```

```
const status = <p style={styles}>Status: {isActive ? 'Active' :  
'Inactive'}</p>;
```

Common camelCase conversions:

background-color becomes backgroundColor

font-size becomes fontSize

font-weight becomes fontWeight

margin-top becomes marginTop

padding-left becomes paddingLeft

border-radius becomes borderRadius

Inline CSS is useful for dynamic styling where styles depend on state or props. However, for static styles, it is often better to use CSS classes for better performance and maintainability.

Combining inline CSS with classes:

```
text
```

```
<div className="container" style={{ backgroundColor: 'blue', padding: '20px'  
}}>
```

```
<p>Content</p>
```

```
</div>
```

While inline CSS works in JSX, it has limitations compared to regular CSS.

You cannot use pseudo-classes like :hover or :focus, media queries, or

animations with inline styles. For these features, you should use CSS classes, CSS modules, or styled-components.